

Lecture 05: Array in JavaScript

Why Arrays Exist: The Core Problem

Declaring independent variables for similar data is inefficient and difficult to manage.

```
let marks1 = 80;  
let marks2 = 90;  
...  
let marks100 = 100;
```

What is an Array?

An **Array** is a non-primitive data type that acts as an ordered collection of elements stored under a single variable name. It provides a structured way to manage lists of data efficiently.

Creating Arrays:

Array Literals

```
let arr1 = [10, 20, 30, 40, 50];
```

Array Constructor

```
let arr2 = new Array(10, 20, 30, 40, 50);
```

The .length Property: Returns the total number of elements present in the array.

```
let marks = [10, 20, 30, 40, 50];  
console.log(marks.length);
```

Accessing Individual Elements: Elements are accessed or modified using zero-based indexing.

```
let marks = [10, 20, 30, 40, 50];  
marks[0] = 15;
```

Heterogeneous Nature: JavaScript arrays are heterogeneous. They can store numbers, strings, booleans, and even other objects or arrays simultaneously:

```
let user = [10, 20, "Rohit", true];
```

Core Array Methods:

.push(element): Adds one or more elements to the end of the array.

.pop(): Removes the last element of the array.

```
let num = [10, 20, 30, 40, 50];  
num.push(60, 70);  
num.pop();
```

.unshift(element): Adds one or more elements to the beginning of the array.

.shift(): Removes the first element of the array.

```
let nums = [30, 40, 50, 60, 70];  
nums.unshift(10, 20);  
nums.shift();
```

Array Iteration:

1. The Classic for Loop:

```
let num = [10, 20, 30, 40, 50];  
  
for(let i = 0; i < num.length; i++) {  
  console.log(num[i]);  
}
```

2. The for...of Loop:

```
let num = [10, 20, 30, 40, 50];  
  
for(let x of num) {  
  console.log(x);  
}
```

Slice vs. Splice:

.slice(startIndex, endIndex): Extracts a section of an array and returns it as a new array without modifying the original array. The endIndex is exclusive.

```
let score = [10, 20, 30, 40, 50, 60, 70];  
console.log(score.slice(2, 4));
```

.splice(startIndex, deleteCount, itemsToInsert): Changes the contents of an array by removing, replacing, or adding elements in place. This mutates the original array.

```
let score = [10, 20, 30, 40, 50, 60, 70]  
score.splice(2, 2);  
score.splice(2, 0, 30, 40);
```

Copy by Reference Mechanics:

Because arrays are non-primitive data types, variables store a memory address (reference) rather than the actual values directly.

```
let arr1 = [0, 20, 30, 40, 50];  
let arr2 = arr1;
```

Note: Before modern ES6 features were introduced, `.concat()` was the primary way to merge two or more arrays into a single, unified array. It does not mutate the original arrays. Instead, it returns a completely brand-new array containing the combined elements.

Modern Operators: Spread, Rest, and Destructuring

The Spread Operator (...): Expands the individual elements of an existing array. This is commonly used to perform shallow copies of array layers or to merge arrays cleanly.

```
let num1 = [10, 20];  
let num2 = [30, 40];  
const num = [...num1, ...num2];
```

Destructuring Assignment: A fast syntax pattern to unpack distinct values from arrays into standalone variables.

```
let num = [10, 20, 30, 40];  
const [first, second] = num;
```

The Rest Operator (...): Used inside a destructuring pattern to gather all remaining unallocated items into a brand-new sub-array.

```
let num = [10, "Rohit", "Aditya"];  
const [digit, ...names] = num;
```

Utility and Search Methods:

.join(separator): Concatenates all elements of an array into a single string, separated by the specified separator string.

.indexOf(element): Returns the first index at which a given element can be found in the array, or -1 if it is not present.

.includes(element): Returns a boolean value indicating whether an array contains a certain element among its entries.

```
const names = ["Hello", "Coder", "Army"];
console.log(names.join(" "))
console.log(names.indexOf("Hello"));
console.log(names.indexOf("Coder"));
```

Multidimensional Arrays:

2D Arrays:

A 2D array is an array whose elements are themselves 1D arrays. The `.length` property of a 2D array returns the total number of nested rows (the inner arrays).

2D Array Iteration:

1. Standard Nested for Loops:

```
let arr = [[1,2,3],[4,5,6],[7,8,9]];

for(let row = 0; row < arr.length; row++) {
  for(let col = 0; col < arr[row].length; col++) {
    console.log(arr[row][col]);
  }
}
```

2. Nested for...of Loops:

```
let arr = [[1,2,3],[4,5,6],[7,8,9]];

for (let row of arr) {
  for (let col of row) {
    console.log(col);
  }
}
```

3D Arrays:

A 3D array adds a third layer of depth, representing an array of 2D arrays.

```
let arr = [[[1, 2, 3],[4, 5, 6]],[[7, 8, 9],[10, 11, 12]]];
```